



Python Virtual Environments

Michael Inden

Freiberuflicher Trainer und Autor



<https://github.com/Michaeli71/Python-Workshop>

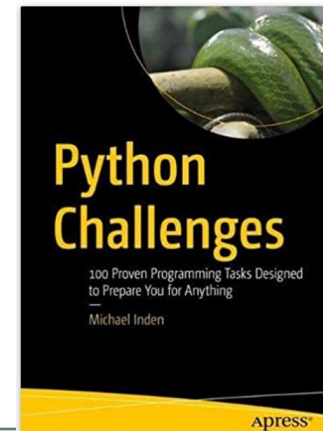
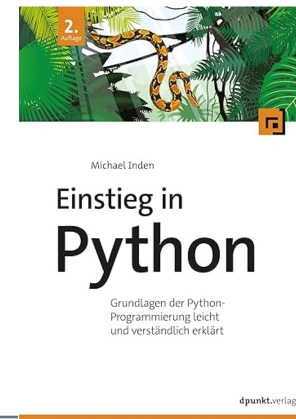
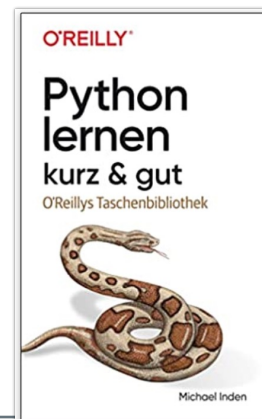
Speaker Intro



- **Michael Inden**, Jahrgang 1971
- Diplom-Informatiker, C.v.O. Uni Oldenburg
- ~8 ¼ Jahre **SSE** bei Heidelberger Druckmaschinen AG in Kiel
- ~6 ¾ Jahre **TPL, SA** bei IVU Traffic Technologies AG in Aachen
- ~4 ¼ Jahre **LSA / Trainer** bei Zühlke Engineering AG in Zürich
- ~3 Jahre **TL / CTO** bei Direct Mail Informatics / ASMIQ in Zürich
- ~3 ½ Jahre **Head of Development** bei Adcubum in Zürich
- Freiberuflicher **Consultant, Trainer** und **Konferenz-Speaker**
- Autor und Gutachter bei dpunkt.verlag, O'Reilly und APress

E-Mail: michael_inden@hotmail.com

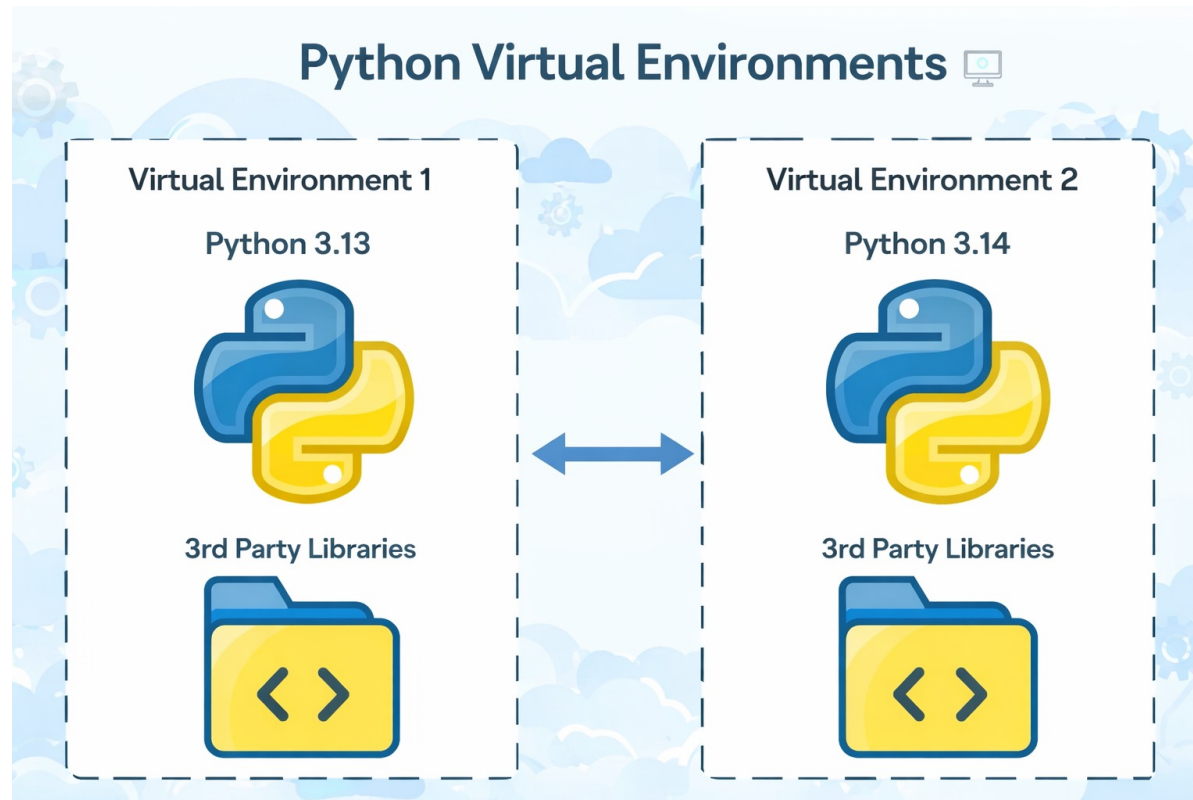
Kurse: **Bitte spricht mich an!**



<https://github.com/Michaeli71/Python-Workshop>



Virtual Environments





- Eine virtuelle Umgebung in Python ist eine isolierte Umgebung, in der man Python-Projekte unabhängig voneinander ausführen und testen kann.

- Python-Basis-Installation bildet die Grundlage und befindet sich hier:

```
$ which python3  
/Library/Frameworks/Python.framework/Versions/3.14/bin/python3
```

- Ist eine Kopie einer Python-Distribution, also Python-Interpreter und Softwarebibliotheken und Binärdateien als Basis für ein eigenes Projekts.
 - Damit lassen sich projektspezifische Abhängigkeiten verwalten, ohne andere Projekte oder die ursprüngliche Python-Installation zu beeinträchtigen.
-



- Wird in einem Verzeichnis, üblicherweise `.venv` oder `venv` im Root des Projektverzeichnis abgelegt
 - Enthält keinen Programmcode
 - Wird nicht in Git verwaltet
 - Kann jederzeit (oder sollte es) komplett neu erstellbar sein
- ```
python -m venv /path/to/new/virtual/environment
```
- Sollte nicht kopiert, sondern an anderer Stelle neu erstellt werden.
  - Lässt sich einfach im Dateisystem löschen, falls obsolete geworden.
-



**Die Verwendung virtueller Umgebungen ist wichtig, weil:**

- sie Paketversionskonflikte zwischen Projekten verhindert
- Projekte portabler und reproduzierbarer macht
- die Python-Installation des Systems sauber hält
- das Testen mit verschiedenen Python-Versionen ermöglicht

**Umfangreiche Dokumentation:**

<https://docs.python.org/3/library/venv.html>

---



- **Virtuelle Umgebung anlegen – `python3 -m venv first_playground`**

```
$ python3 -m venv first_playground
$ python3
Python 3.14.2 (v3.14.2:df793163d58, Dec 5 2025, 12:18:06) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more
information.
>>>
```

- **Auf ein externes Modul zugreifen ... (natürlich geht das nicht 😊)**

```
>>> import cowsay
Traceback (most recent call last):
 File "<python-input-0>", line 1, in <module>
 import cowsay
ModuleNotFoundError: No module named 'cowsay'
```





- Virtuelle Umgebung aktivieren

| Platform | Shell      | Command to activate virtual environment                   |
|----------|------------|-----------------------------------------------------------|
| POSIX    | bash/zsh   | <code>\$ source &lt;venv&gt;/bin/activate</code>          |
|          | fish       | <code>\$ source &lt;venv&gt;/bin/activate.fish</code>     |
|          | csh/tcsh   | <code>\$ source &lt;venv&gt;/bin/activate.csh</code>      |
|          | pwsh       | <code>\$ &lt;venv&gt;/bin/Activate.ps1</code>             |
| Windows  | cmd.exe    | <code>C:\&gt; &lt;venv&gt;\Scripts\activate.bat</code>    |
|          | PowerShell | <code>PS C:\&gt; &lt;venv&gt;\Scripts\Activate.ps1</code> |

- Also machen wir das mal – `source first_playground/bin/activate`

```
$ source first_playground/bin/activate
(first_playground) michaelinden@MacBook-Pro-M3-MAX-von-Michael ~ %
$ which python3
/Users/michaelinden/first_playground/bin/python3
```





- **pip ist der Paketmanager für Python und wird verwendet, um zusätzliche Bibliotheken und Tools zu installieren.**
  - **Mit pip lassen sich Pakete einfach aus dem Python Package Index (PyPI) herunterladen und verwalten. (vergleichbar Maven Central)**
  - **pip ermöglicht das Installieren, Aktualisieren und Entfernen von Python-Paketen per Kommandozeile.**
  - **Abhängigkeiten von Projekten können mit pip automatisch mitinstalliert werden.**
  - **pip ist bei modernen Python-Versionen standardmäßig enthalten und sofort einsatzbereit.**
-



- **Installation eines einfachen externen Moduls**

```
$ which pip
/Users/michaelinden/first_playground/bin/pip
(first_playground) michaelinden@MacBook-Pro-M3-MAX-von-Michael ~ % pip install cowsay
Collecting cowsay
 Using cached cowsay-6.1-py3-none-any.whl.metadata (5.6 kB)
Using cached cowsay-6.1-py3-none-any.whl (25 kB)
Installing collected packages: cowsay
Successfully installed cowsay-6.1
```

- **Python starten und Import**

```
$ python3
Python 3.14.2 (v3.14.2:df793163d58, Dec 5 2025, 12:18:06) [Clang 16.0.0 (clang-
1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import cowsay
```

# Zugriff auf das installierte Modul



- **Aufruf des externen Moduls**

```
>>> cowsay.cow("Hello IVV")
```

```
| Hello IVV |
=====
```

ASCII art of a cow:

```
 ^ ^
 (oo)_____
 (____)_____)\\/
 ||----w |
 || ||
```

- **Umgebung deaktivieren – deactivate**

```
>>> exit
(first_playground) michaelinden@MacBook-Pro-M3-MAX-von-Michael ~ % deactivate
— michaelinden@MacBook-Pro-M3-MAX-von-Michael ~ %
```

# Rückschalten auf Standardinstallation



- Dort ist das Modul erwartungskonform nicht enthalten

```
michaelinden — Python — 150x52

[michaelinden@MacBook-Pro-M3-MAX-von-Michael ~ % clear]

[michaelinden@MacBook-Pro-M3-MAX-von-Michael ~ % which python3]
/Library/Frameworks/Python.framework/Versions/3.14/bin/python3

[michaelinden@MacBook-Pro-M3-MAX-von-Michael ~ % python3]
Python 3.14.2 (v3.14.2:df793163d58, Dec 5 2025, 12:18:06) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.

[>>> import cowsay]
Traceback (most recent call last):
 File "<python-input-0>", line 1, in <module>
 import cowsay
ModuleNotFoundError: No module named 'cowsay'
>>>
```



- **Python 2 unterstützt virtuelle Umgebungen nicht nativ. Daher werden hierfür externe Werkzeuge benötigt. Eines der bekanntesten ist virtualenv.**
- **Mit virtualenv wird eine eigene Verzeichnisstruktur erstellt, in der alle benötigten Python-Dateien und Abhängigkeiten isoliert abgelegt werden. Die Installation erfolgt über `pip install virtualenv`. Eine virtuelle Umgebung wird anschließend mit `virtualenv <Verzeichnis>` angelegt.**
- **Das Aktivieren und Deaktivieren der virtuellen Umgebung funktioniert genauso wie bei Python 3.**
- **Da Python 2 nicht mehr weiterentwickelt wird, sollten virtuelle Umgebungen unter Python 2 ausschließlich für die Wartung bestehender Legacy-Projekte verwendet werden.**



- **Anaconda ist eine kostenlose Python-Distribution, die besonders für Data Science, Machine Learning und wissenschaftliches Rechnen geeignet ist.**
- **Sie enthält Python sowie viele wichtige Bibliotheken wie NumPy, Pandas, Matplotlib und SciPy bereits vorinstalliert.**
- **Mit dem Paket- und Umgebungsmanager conda lassen sich Abhängigkeiten und virtuelle Umgebungen einfach verwalten.**
- **Tools wie Anaconda Navigator und Jupyter Notebook erleichtern das Arbeiten, Experimentieren und Visualisieren von Daten.**
- **Anaconda ist plattformübergreifend (Windows, macOS, Linux) und ideal für Einsteiger wie auch für fortgeschrittene Nutzer.**





---

# Wissenswertes rund um pip und virtuelle Environments





# Überblick über installierte Module



- **pip/pip3 list**

| Package              | Version    |
|----------------------|------------|
| -----                | -----      |
| annotated-types      | 0.7.0      |
| anyio                | 4.11.0     |
| appnope              | 0.1.4      |
| argon2-cffi          | 25.1.0     |
| argon2-cffi-bindings | 25.1.0     |
| arrow                | 1.4.0      |
| asttokens            | 3.0.1      |
| async-lru            | 2.0.5      |
| attrs                | 25.4.0     |
| autogen-agentchat    | 0.7.5      |
| autogen-core         | 0.7.5      |
| babel                | 2.17.0     |
| beautifulsoup4       | 4.14.3     |
| bleach               | 6.3.0      |
| certifi              | 2025.11.12 |
| cffi                 | 2.0.0      |
| charset-normalizer   | 3.4.4      |
| comm                 | 0.2.3      |
| ...                  |            |

## In der virtuellen Umgebung

```
(first_playground) michaelinden@MacBook-Pro-M3-MAX-von-Michael ~ % pip list
Package Version

cowsay 6.1
pip 25.3
```



# Auflistung aller installierten Module



- **pip/pip3 freeze**

## In der virtuellen Umgebung

```
annotated-types==0.7.0
anyio==4.11.0
appnope==0.1.4
argon2-cffi==25.1.0
argon2-cffi-bindings==25.1.0
arrow==1.4.0
asttokens==3.0.1
async-lru==2.0.5
attrs==25.4.0
autogen-agentchat==0.7.5
autogen-core==0.7.5
babel==2.17.0
beautifulsoup4==4.14.3
bleach==6.3.0
certifi==2025.11.12
cffi==2.0.0
charset-normalizer==3.4.4
comm==0.2.3
contourpy==1.3.3
cyclers==0.12.1
...
```

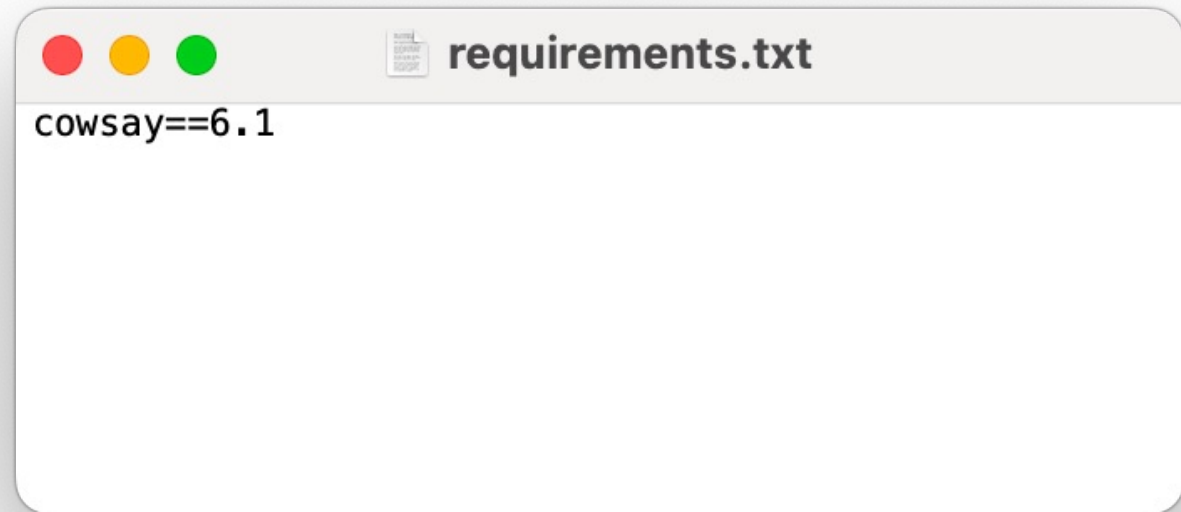
```
[(first_playground) michaelinden@MacBook-Pro-M3-MAX-von-Michael ~ % pip freeze
cowsay==6.1
```

# Bereitstellung einer Liste aller installierten Module



- **pip/pip3 freeze > requirements.txt**

```
annotated-types==0.7.0
anyio==4.11.0
appnope==0.1.4
argon2-cffi==25.1.0
argon2-cffi-bindings==25.1.0
arrow==1.4.0
asttokens==3.0.1
async-lru==2.0.5
attrs==25.4.0
autogen-agentchat==0.7.5
autogen-core==0.7.5
babel==2.17.0
beautifulsoup4==4.14.3
bleach==6.3.0
certifi==2025.11.12
cffi==2.0.0
charset-normalizer==3.4.4
comm==0.2.3
contourpy==1.3.3
cycller==0.12.1
...
```





- **Möglichkeiten zur Spezifikation von Abhängigkeiten**

```
Requirements without Version Specifiers
nose
nose-cov
beautifulsoup4
#
Requirements with Version Specifiers
See https://www.python.org/dev/peps/pep-0440/#version-specifiers
docopt == 0.6.1 # Version Matching. Must be version 0.6.1
keyring >= 4.1.1 # Minimum version 4.1.1
coverage != 3.5 # Version Exclusion. Anything except
version 3.5
Mopidy-Dirble ~= 1.1 # Compatible release. Same as >= 1.1, ==
1.*
```

# Installation basierend auf requirements.txt

---



## 1. Neue virtuelle Umgebung erstellen

```
$ python3 -m venv second_playground
```

```
(second_playground) michaelinden@MacBook-Pro-M3-MAX-
von-Michael ~ % pip list
Package Version

pip 25.3
```

## 2. Requirements-Datei dorthin kopieren

## 3. Neue Umgebung aktivieren

```
$ source first_playground/bin/activate
```

## 4. Installation der Abhängigkeiten mit pip und diesem Kommando:

```
$ pip install -r /path/to/requirements.txt
```

---

# Installation basierend auf requirements.txt



## 1. Protokollierung der Installation

```
(second_playground) michaelinden@MacBook-Pro-M3-MAX-von-Michael second_playground % pip install -r ./requirements.txt
Collecting cowsay==6.1 (from -r ./requirements.txt (line 1))
 Using cached cowsay-6.1-py3-none-any.whl.metadata (5.6 kB)
Using cached cowsay-6.1-py3-none-any.whl (25 kB)
Installing collected packages: cowsay
Successfully installed cowsay-6.1
```

## 2. Prüfe mit pip list:

```
(second_playground) michaelinden@MacBook-Pro-M3-MAX-von-Michael second_playground % pip list
Package Version

cowsay 6.1
pip 25.3
```



- **PyCharm und requirements.txt**
    - <https://www.jetbrains.com/help/pycharm/managing-dependencies.html#sync-requirements>
  - **Tool uv als sehr performante Alternative zu pip**
    - <https://www.youtube.com/watch?v=6pttmsBSi8M>
    - <https://www.youtube.com/watch?v=AMdG7IjgSPM>
  - **Tool conda als Alternative zu venv**
    - <https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html>
-



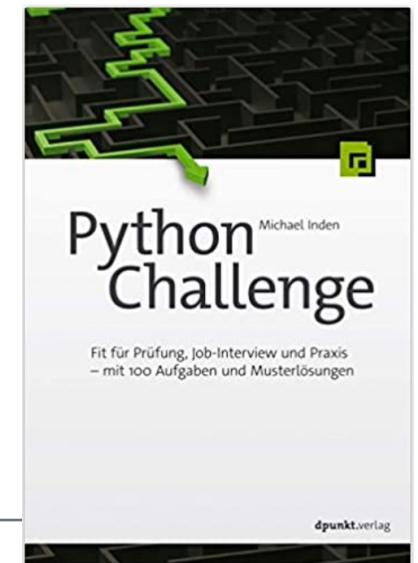
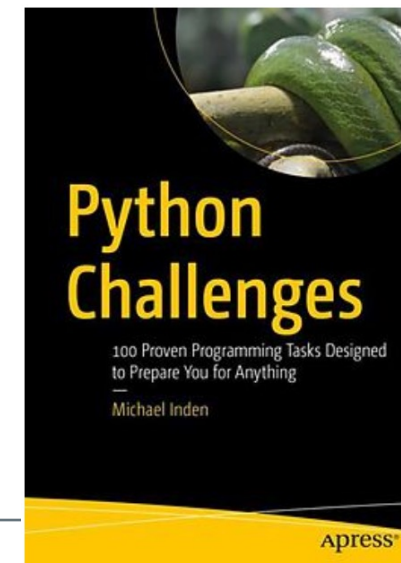
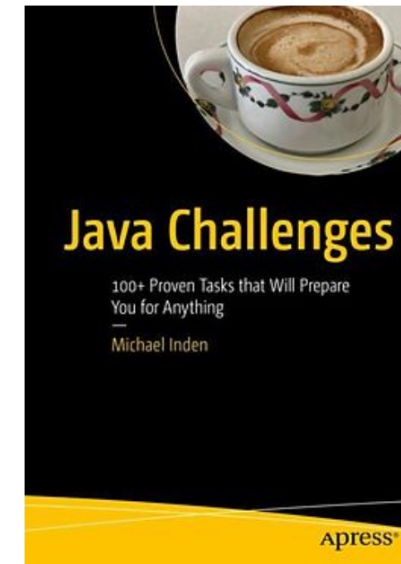
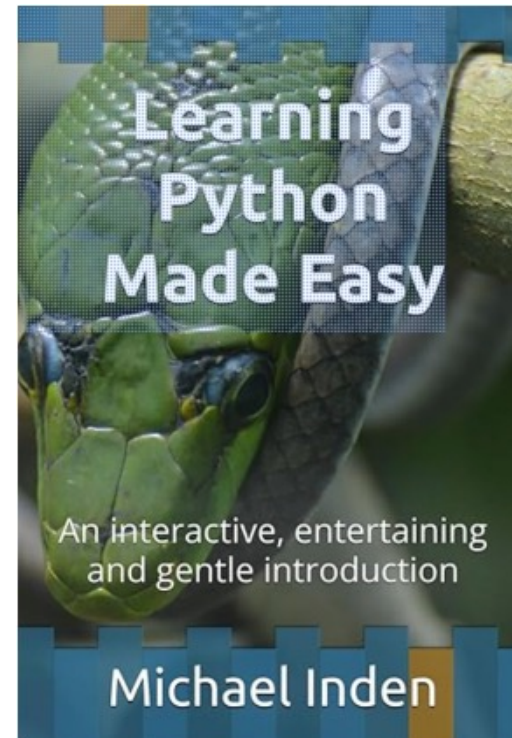


---

# Questions?

---

Help





---

# Thank You

---